1.1–1.2 · What this course is, and the stack we will fill in all semester

Sessions: 1.1 (what "learning for robotics" means here, why now, how the course runs), 1.2 (what manipulation is; the manipulation stack as the map of the semester). **Teaching day:** Deck 01, the single teaching day of week 1, one 100-minute block. **Prerequisites established earlier:** none; this is the first day. **Notation used (preview only, nothing is derived today):** `x_t` robot state, `q` joint positions, `u_t` control, `o_t` observation, `φ(q)` forward kinematics, `π` policy, `ℓ` language instruction, `y*` task-space target. All are in `notation.md` §2.1, §2.3, §2.5. No collisions: the RL-side `a` is not used today (ruling R4; the bridge slide comes in 6.4).

0. What you will be able to do afterwards

1. Name the five components of a classical manipulation stack (perception, goal, reference motion, controller, motors), and say for each what it takes in and what it puts out.
2. Place a robot-learning result you read about (a vision-language-action model, a learned grasp detector, a reinforcement-learned locomotion policy) on that stack: which component it replaces, which it keeps, and which it silently assumes.
3. State how the course runs (calendar rhythm, three assignments, project, midterm, no final), and install the simulator before the reserved day R 3.

1. Motivation and analogy

Suppose a friend gives you an address in a city you have never visited and asks you to drive there. You will do five distinct things, whether or not you notice them.

1. You **look**: you read street signs and see where the other cars are. This is *perception*: turning raw sensor data into an estimate of where things are.
2. You **decide where to end up**: not "the address" as a string, but a physical spot, a parking space with the car pointing the right way. This is the *goal*: converting a task ("be at 12 Cherry Street") into a configuration of the vehicle.
3. You **plan a route** from where you are to that spot that avoids one-way streets and closed roads. This is the *reference motion*: a path through the space of allowed configurations.
4. You **steer and press the pedals** so the car actually follows the route, correcting when a gust of wind pushes you sideways. This is the *controller*: a fast loop that maps the current state to the next command.
5. The **engine and wheels** turn commands into motion. These are the *motors*.

Every one of these steps uses a *map*: a model of the city that says which streets exist, how they connect, and how long the car is. In robotics the map is the *model* of the robot and its surroundings: link lengths, joint limits, the shapes of objects, the physics of contact. The classical half of this course (weeks 1–5) builds exactly this five-step stack for a robot arm, and every step is written as mathematics we can check line by line.

Now imagine a second driver: a taxi driver who has driven this city for twenty years and never opens a map. They hear the address and simply go. They are faster than you, they do not stop to plan, and they handle the busy roundabout without thinking. This is a *learned policy*: a function trained on many past drives that maps what it sees and hears directly to steering. The learned half of this course (weeks 5–12) studies these drivers: how they are trained (imitation, then reinforcement), what they get right, and what they silently assume. The taxi driver assumes the city has not changed. They cannot show you the route before driving it, and they cannot explain why they turned left.

Where the analogy breaks down. A car is one rigid body with two or three controls, and it never touches anything on purpose. A robot arm has seven joints, and its whole reason for existing is to *touch things*: to grasp, push, and carry. The moment the gripper closes on a block, the block becomes part of the arm, and the "map" changes: a new body is attached, new collisions matter, and the goal is now defined in terms of an object, not the robot. There is no driving equivalent of the road becoming part of your car. This break is not a weakness of the analogy but the central difficulty of manipulation, and we return to it in session 2.1 as the reason model-based methods struggle at exactly the point where manipulation begins.

2. Prerequisite ramp

Not applicable today: no new mathematical tool appears. The course assumes three things, and you should check them against yourself this week.

- **Linear algebra.** You can multiply a 3×3 matrix by a vector by hand, you know what the transpose and the inverse are, and you know what it means for a matrix to be orthogonal. Week 2 uses all of this for rotations and transforms.
- **Probability.** You know what an expectation and a conditional distribution are. Nothing more is needed until week 6.
- **Python with numpy.** You can write a function that takes an array and returns an array, and run it from a script or a notebook. All three assignments are Python.

Nothing about robotics, control, optimization, or deep learning is assumed. Where such a tool is needed, the course introduces it first: optimization in weeks 3–4, Markov decision processes in week 6, neural policies in week 7.

3. Formal setup: the stack, and the symbols each component carries

A manipulation system is a loop that runs many times per second. We name its parts now and fill them in over the semester; today only the names and the interfaces matter.

Component	Input	Output	Where it is taught (classical)	Learned counterpart
Perception / state estimation	observation `o_t` (camera images, joint encoders)	estimate of the state: robot joint positions `q_t`, object poses `^W X^O`	5.2, 6.1	6.2 learned keypoints and grasps; 6.3 open-vocabulary perception with a vision-language model (VLM)
Goal (inverse kinematics, IK)	task-space target `y*` (where the gripper should be)	a final joint configuration `q_T` with `φ(q_T) = y*`, where `φ` is the forward-kinematics map	2.2–2.4, 3.1–3.2, 4.2	5.1 learned samplers and feasibility; 7.1 a policy that fulfils the constraint directly
Reference motion (planning)	`q_0`, `q_T`, the obstacles	a collision-free trajectory `q_{0:T}`	4.2 trajectory optimization (KOMO), 4.4 sampling-based planning (RRT, PRM)	5.1 learned heuristics for planners; 7.2 generative policies that output motions

Component	Input	Output	Where it is taught (classical)	Learned counterpart
Controller	current state `x_t = (q_t, q̇_t)` and the reference	control `u_t` (joint torques or commanded accelerations)	4.3	8.1 a learned dynamics model with a planner inside it (learned model + MPC); 8.3–9.1 a vision-language-action model (VLA) emits action chunks; 10.4–11.2 reinforcement learning tunes the policy
Motors	`u_t`	physical motion	not taught (mechanical design is out of scope)	—

Two remarks on symbols, so that nobody has to unlearn anything later.

- The **state** `x_t` is the state of the robot *and* of the environment (deck 01 p.21). A policy in the classical stack is a map `π : x_t ↦ u_t`. A learned policy usually does not see `x_t`; it sees the observation `o_t`, and for a VLA also a language instruction `ℓ`: `π_θ(a_{t:t+H} | o_t, ℓ)`, where `θ` are the learned parameters and `a_{t:t+H}` is a *chunk* of `H` future actions. We write robot-side controls as `u` and actions inside the learning blocks as `a`; session 6.4 has one explicit slide that says `a ≡ u` when the learner is a robot (notation ruling R4).
- The **map** of the analogy is the set of models the classical stack needs: the kinematic structure (link lengths, joint types), the shapes of every body (for collisions), and, once dynamics enter, masses and friction. Deck 03 p.65 lists exactly this as "what we assumed to know", and session 2.1 asks which of these assumptions learning can remove.

4. Derivation

Not applicable today. The first derivation of the course is the 2-D rotation matrix from the angle-sum formulas in session 2.2, and the first one that matters for an assignment is the damped Gauss-Newton step for inverse kinematics in session 3.2.

5. Worked example: one pick-and-place through the five boxes

The numbers below are illustrative and rounded; they are chosen to be the right order of magnitude for a Franka Panda arm on a table, and the code in §6 does not reproduce them (today's code only checks the installation). From week 2 on, every worked example is reproduced by the code of the day.

Task. "Pick up the red block." The robot is a 7-joint arm; the block is a 6 cm cube.

1. **Perception.** The camera returns an image `o_t`. The perception component (week 5–6) segments the red region, back-projects it to a point cloud, and estimates the block's pose relative to the world frame: position `^W p^Box = (0.50, 0.10, 0.72)` m, rotated 30° about the vertical axis. It also reads the robot's joint encoders: `q_t`, seven numbers in radians.
2. **Goal.** We want the gripper frame `G` to be 10 cm above the block, pointing down, before we descend. That is a target pose `y*` for `G`. Inverse kinematics (weeks 2–4) searches for joint angles `q_T` such that `φ(q_T) = y*`, staying inside the joint limits and away from the table. The answer is one configuration, seven numbers, out of the infinitely many that reach the same pose; the optimization picks the one closest to the current posture.
3. **Reference motion.** From `q_0` (now) to `q_T` (pre-grasp), we want a smooth, collision-free trajectory over, say, `T = 2` seconds, discretised into 100 steps: `q_{0:100}`. Trajectory optimization (KOMO, week 4) minimises jerk subject to "no collision" at every step; a sampling planner (RRT, week 4) would instead grow a random tree of collision-free configurations and smooth the path it finds.
4. **Controller.** A loop at 1000 Hz reads `x_t = (q_t, q̇_t)`, compares it with the reference at time `t`, and outputs joint torques `u_t` that pull the arm back onto the reference (proportional-derivative control, week 4). Between two planner updates the controller runs a thousand times.
5. **Motors.** The torques move the arm. Then we descend 10 cm, close the gripper, lift.

What changes at the grasp. As soon as the fingers close, the block is a new body rigidly attached to the gripper. Collision checking must now include the block; the goal for "place the block on the shelf" is a pose of the *block* frame, which we get from the gripper pose by composing transforms (`^W X^Box = ^W X^G · ^G X^Box`, week 2). If the block slips in the grasp by 1 cm, every downstream computation is 1 cm wrong and nothing in the stack notices unless perception looks again. This is the analogy break of §1, made concrete.

The learned version of the same task. A VLA receives the camera image and the string "pick up the red block" and outputs a chunk of `H = 50` joint-position targets covering the next second, then does it again. There is no explicit block pose, no `q_T`, no planner: components 1–3 are replaced by one network, and component 4 shrinks to a position controller that tracks the emitted targets. What the network silently

assumes — that the scene resembles its training data, that the camera has not moved, that "red block" means what it meant in training — is what assignment A3 is designed to expose.

6. Code: `code/hello_rai.py`

The code for Day 1 is the installation check for reserved day R 3. It loads the single-arm Panda scene shipped with RAI (the `robotic` package), prints the number of frames and joints, prints the gripper pose relative to the world frame as a position and a rotation, adds a box as a child of the table frame, prints the box's world position (RAI composing the transform for us), and opens the viewer.

```
python -m venv .venv && source .venv/bin/activate
pip install robotic numpy
python lectures/01-D1-intro-and-the-stack/code/hello_rai.py
```

Environment and platform (checked 2026-09-04). The `robotic` package publishes wheels for Linux x86_64 only (version 0.4.1 on that date). Linux and Windows through WSL work with the commands above; in Spring 2026 every student used one of these two. On macOS there is no wheel; the options are Docker with `--platform linux/amd64`, a Linux lab machine, or a Colab notebook (everything except the interactive viewer works there). A cross-platform Drake version of the course examples is being prepared for a later semester (`drake/README.md`). Expected runtime: under two seconds plus the viewer.

Expected output. To be recorded here from the R 3 dry run on a Linux machine. On the authoring machine (macOS, Apple silicon) the package cannot be installed natively and Docker was not running when these notes were written, so the numbers are deliberately left blank rather than guessed. The frame count and the seven joint values are the two lines students should compare against.

7. Common failure modes (today: misconceptions about the course)

1. **"This is a deep-learning course with robots as the dataset."** It is not. Weeks 1–5 build the classical stack in full, and assignment A1 contains no neural network. A student who skips the classical half will be unable to say what a learned policy has replaced, which is learning outcome 2 of the syllabus.
2. **"This is a robotics course and the learning is a coda."** It is not that either. The learned counterpart of each classical component appears from week 5, VLA anatomy is in week 8, and A3 is a VLA assignment. The two halves are interleaved on purpose.

3. **"VLAs make the classical stack obsolete, so why learn it."** Three answers. First, you cannot evaluate a policy without knowing what it replaced and what it assumes. Second, today's VLAs still run inside a stack: a low-level controller tracks their outputs, and their inference latency is handled by engineering around the controller box (real-time chunking, week 9). Third, the field's own recent turn to reinforcement learning as *post-training* of VLAs (`literature/vla-literature-2026-09-04.md` §4) is a return to the optimal-control view of the classical stack.
4. **"I need robotics or deep-learning background already."** Only linear algebra, probability, and Python are assumed (§2). Both halves of the audience will find one half of the course new; that is intended.
5. **"The assignments are installation exercises."** Last year's honest reviews (deck 01 p.24) said the simulator was the obstacle. This year: the install check above is given on Day 1, R 3 is a hands-on RAI tutorial before anything is graded, and each handout states the tested platform. Installation problems are reported on the forum, not solved alone.

8. Where this sits in the stack

Today *is* the stack. Every later day names its box in this table (§3) and, for classical material, the learned component that replaces it later, or, for learned material, the classical component it replaces and what it silently assumes. The table is reproduced as slide 27 and reused in sessions 2.1, 5.1, 6.2, 7.1, and 8.3.

9. Further reading (optional, never load-bearing)

- Matthew T. Mason, *Toward Robotic Manipulation*, Annual Review of Control, Robotics, and Autonomous Systems 1:1–28, 2018. The five definitions of manipulation (slide 16).
- Russ Tedrake, *Robotic Manipulation*, chapter 1 (manipulation.csail.mit.edu). The "anatomy of a manipulation system" that our stack diagram follows; this course borrows Drake's monogram notation from week 2 on.
- Marc Toussaint, *Robot Learning* lecture script, sections 1–3 (Summer 2024): the taxonomy of what is learned and how data is generated, and the "which component does learning shortcut" framing that session 2.1 uses.
- `literature/vla-literature-2026-09-04.md` §1 and §4 for every dated claim on slide 5.
- Daniel Wolpert, *The real reason for brains*, TED 2011 (the quote on slide 6).

Open questions and weak points

- **Quiz count.** The confirmed grading scheme has five quizzes (16 %); the syllabus draft had two. The five are now placed on the five non-midterm reserved days (slide 12–13), which puts quiz 1 on R 3 after only three teaching days, and quiz 3

on R 7, which falls on the 28 October half day. Both placements are provisional.

- **Videos.** Day 1 now carries eight clips and every one is attached (2026-09-15). External, as clickable thumbnail plus caption link: the shoelace knot (slide 17), the crow's eight-stage puzzle and Stoffel the honey badger (slide 18), Moley's kitchen (slide 20), and the Sollerman hand function test (slide 22, added by the instructor). Local files in `fig/`, played with `<video>`: `tri_allegro_plate_pick.mp4` beside the shoelace on slide 17, `tri_mmt_shop.mp4` on slide 20, and `tri_fails.mp4` on the new slide 21. The external links are live clickable annotations in `slides.pdf`; the local clips play only in the HTML export (see the next bullet). Nothing on Day 1 is now promised and missing.
- **A clip kept in Google Drive.** Same convention as a YouTube clip: a thumbnail wrapped in a link with `target="_blank"`, so a click opens the file in Drive's own player in a new tab. The file must be shared as "anyone with the link"; the thumbnail comes from `https://drive.google.com/thumbnail?id=<FILEID>&sz=w1200` (works only for such files, which is also a quick check of the sharing setting). Slides 26 and 31 use this (the pavilion assembly clip; two student project clips).
- **Local video files.** Tested 2026-09-15 (the note in `README.md` had not been re-tested since the theme changed). A local clip is written as `<video src="fig/clip.mp4" poster="fig/clip-still.jpg" controls muted loop width="620"></video>` and the file goes in this deck's `fig/`. In the HTML export (`HTML=1 scripts/render_slides.sh`) it plays inline; the `src` stays a relative path, so `fig/` must travel with `slides.html`. In `slides.pdf` it cannot play: the PDF shows the poster **with dead player controls drawn on top**, and a loading spinner if the file is missing — so for the PDF an animated GIF via `![w:620 center](fig/clip.gif)` looks better, since the PDF shows its first frame cleanly as an ordinary figure. Keep clips small: nothing in `.gitignore` excludes video, and a committed clip stays in the repository's history for ever.
- **The analogy** is a proposal awaiting the instructor's review. If another is preferred, §1 and slides 26–27 change; nothing else does.
- **`hello_rai.py` is unverified** on 2026-09-04 (no Linux runtime on the authoring machine). Its API calls follow the current RAI tutorials; the R 3 dry run must confirm the frame name `l_gripper`, the `table` frame, and the `view(True, ...)` signature.
- **macOS students.** Last year nobody used a Mac; everyone ran Ubuntu natively or in WSL. Slide 11 says so. A Drake-based, cross-platform copy of the examples is planned for a later semester and is not promised to this cohort.
- **Pacing risk.** Session 1.1 carries fourteen slides of framing and logistics; if it runs long, slides 10 (venues) and 8 (types of robotics) are the ones to skip.
- **Thin pedagogy.** §5's numbers are illustrative, not reproduced by code, which breaks the template's rule for one day. The first fully reproducible example is 2.2's ball on a tray.

Slide plan for this day (32 slides + midpoint, 104 minutes)

#	Slide	Min	Notes section	Source
1	Title; today's two questions	1	—	new
2	What "learning for robotics" means here; what we cover	3	§0	deck 01 p.3–4
3	What we do not cover; the CS 687 boundary	2	§7	deck 01 p.3–4
4	The thesis of the course: build the stack, then learn what replaces each part	3	§1, §8	new
5	Why now (dated 2026-09-04)	4	§7.3	literature §1, §4
6	Motion as the driver of intelligence	3	§1	deck 01 p.19–20, retyped as text; the tunicate is a crop of p.20
7	Robot state × environment state	3	§3	deck 01 p.21, retyped as text
8	Types of robotics; we do manipulation	2	—	deck 01 p.25, retyped as text; the plate strip is a crop of p.25
9	"Nothing more practical than a good theory"; prerequisites; honest reviews	4	§2, §7.5	deck 01 p.24, retyped as text (reviews transcribed)

#	Slide	Min	Notes section	Source
10	Where the field talks	2	§9	deck 01 p.18, 58
11	Tools we use and why	3	§6	deck 01 p.49–50 + new
12	Logistics: grading, F-rules, midterm, no final	4	—	deck 01 p.2
13	The semester rhythm	3	—	syllabus §0
14	The semester on one slide	4	§3	syllabus §2
15	Assignments and project timeline	4	—	syllabus §3–4
—	Midpoint / break	5		
16	What is manipulation? Mason's definitions	3	§1	deck 01 p.10
17	Why hands are hard: a shoelace, and a robot hand on a plate	4	§1	deck 01 p.11; local clip
18	Manipulation without hands: a crow's eight-stage puzzle, a honey badger's improvised tools	4	§1	new; BBC Earth and BBC clips
19	Open-world manipulation needs	3	§5	deck 01 p.12

#	Slide	Min	Notes section	Source
20	Examples I: a scripted kitchen, and a mobile manipulator in a real shop	4	—	deck 01 p.14; local clip
21	The same robot, in the same shop, when it fails	4	§7	new; local clip (Ava's Market field test)
22	Examples II: everyday objects are hard for people too	2	—	deck 01 p.28
23	Examples III: learning inside a classical planner (the group's work)	4	§8	deck 01 p.31, 35, 40
24	Sequential manipulation and task-and-motion planning, pointer to week 13	2	—	deck 01 p.29, retyped as text; the two pictures are crops of p.29
25	The pavilion assembly, executed: a 60 s clip opened from Google Drive	2	—	instructor's Drive (20_Assembly_60sec.mp4), linked 2026-09-16
26	The stack diagram, version 0	4	§3	deck 01 p.53, deck 03 p.12
27	The driving analogy on the boxes, and where it breaks	4	§1	new

#	Slide	Min	Notes section	Source
28	The stack coloured by week	4	§3, §8	new
29	The methods table revisited: the “?” of 2026	3	§7.3	deck 01 p.5, retyped as a four-column text table
30	Takeaways and expectations	2	§0	deck 01 p.59
31	What previous students built: four project clips in a 2×2 grid (two YouTube, two Drive)	3	—	instructor’s links, 2026-09-16
32	Next time, and what to install before R 3	2	§6	new